

QE Automation Framework

A TypeScript Playwright framework built on Page Object Model + Component Object Model, dependency-injected fixtures, environment-driven config, and Allure/HTML reporting. This is the complete walkthrough — every feature, the steps to use it, and what it actually looks like on screen.

Node.js 20+

Playwright 1.61

TypeScript

Allure 3

01

What you're working with

The shape of the framework before you touch a single command.

This repo automates a browser application end to end: UI flows via Page Objects, API calls via a typed client, and everything wired together through Playwright fixtures so a test never constructs its own dependencies. Every run produces two independent reports — Playwright's own HTML report, and a much richer Allure report with named steps, screenshots, and video evidence attached automatically.

Page Object Model

UI locators and actions live in `src/pages/`, never inline in a test.

Fixtures

Every page, service, and helper is injected — tests destructure what they need instead of calling `new`.

Multi-environment

Four config tiers (`qa` / `stage` / `uat` / `prod`) switched with one env var.

8 sample scripts

Integration through visual regression, each a working reference for that reporting pattern.

AI-assisted

Five Copilot chatmodes, two auto-loaded skills, and an MCP server for programmatic control.

Quick start

Five commands from a clean clone to a passing suite.

1 Install dependencies

Also wires up the Husky pre-commit hook (lint-staged runs ESLint + Prettier on every commit).

```
npm install
```

2 Install browsers

Downloads Chromium, Firefox, and WebKit with their OS dependencies.

```
npm run install:browsers
```

3 Configure an environment

Copy the template and fill in real values. `.env.qa` already ships with safe local defaults pointed at the bundled demo app.

```
cp .env.example .env.qa
```

4 Run the suite

`npm test` starts the bundled demo server automatically via Playwright's `webServer` option — nothing to run in a second terminal.

```
npm test
```

5 Open the report

```
npx playwright show-report reports/html
```

No real target app required to start. A minimal demo app ships in `demo/` — a login form, a dashboard, and an animated component — so the smoke/sanity/e2e suites and every sample script run immediately, with zero external dependencies.

Architecture

Where things live, and why.

`src/pages/` sits directly under `src/` because it's what tests touch most. Everything else — the framework "engine" — lives under `src/core/`, grouped by role:

src/core/ui

BasePage, reusable Components, Locators — everything that touches the raw Playwright Page API.

src/core/api

ApiClient, business Services, optional multi-database clients.

src/core/runtime

Fixtures, GlobalSetup/Teardown, console + failure capture hooks.

src/core/data

Test data, Fluent builders, domain models, enums.

src/core/config

The `Config` singleton, constants, Winston logger.

src/core/shared

Interfaces, exceptions, `*Utils` static helpers, validations.

The rule of thumb

- Tests only do **Arrange / Act / Assert**, using fixtures — never `new PageObject(page)`.
- Page objects only do locators, actions, and navigation — **zero assertions**.
- Assertions live in `ValidationHelpers` or a plain `expect()` in the test.
- Nothing is hardcoded — URLs, credentials, and timeouts always route through `Config` or `src/core/config/constants/`.

src/core, file by file

What's actually inside each of the six groups — every file, real.

ui

`src/core/ui/`

Everything that touches the raw Playwright `Page / Locator` API. `BasePage` is the base every page object extends; `BaseComponent` is the same idea scoped to a root locator, for reusable UI fragments. `LocatorFactory` and `CommonLocators` steer every locator toward `getByRole / getByLabel / getByText` — never CSS, never XPath.

<code>base/BasePage.ts</code>	click, fill, hover, drag-drop, upload, download, wait*, screenshot, alert handling — 25+ actions
<code>components/BaseComponent.ts</code>	scopes a component to a root Locator
<code>components/HeaderComponent.ts</code>	logo, nav, profile/logout buttons
<code>components/NavigationMenu.ts</code>	
<code>components/TableComponent.ts</code>	role="row"/"cell" driven, works on any table
<code>components/Pagination.ts</code>	next/previous, page indicator text
<code>components/ToastMessage.ts</code>	role="status" notifications
<code>components/CommonDialog.ts</code>	role="dialog" confirm/cancel modals
<code>locators/LocatorFactory.ts</code>	byRoleAndName, byLabel, byPlaceholder, byText, byTestId
<code>locators/CommonLocators.ts</code>	selectors shared across more than one page

Every `BasePage` action follows the same shape — wait for the element, then act, with debug logging on every call:

```
async click(locator: Locator): Promise<void> {
  Logger.debug(`Clicking locator: ${locator}`);
  await locator.waitFor({ state: 'visible' });
  await locator.click();
}
```

api

`src/core/api/`

Backend I/O. `ApiClient` wraps GET/POST/PUT/PATCH/DELETE with built-in status/JSON assertions; `services/` wrap it again with domain methods (`AuthService.login()`, `JobService.createJob()`); `database/` is an opt-in multi-driver layer behind one factory.

<code>ApiClient.ts</code>	GET/POST/PUT/PATCH/DELETE + expectStatus/getJson
<code>services/AuthService.ts</code>	
<code>services/JobService.ts</code>	
<code>database/DatabaseClientFactory.ts</code>	<code>create(DatabaseType.X, options) -> IDatabaseClient</code>
<code>database/IDatabaseClient.ts</code>	
<code>database/PostgresClient.ts</code>	pg — not installed by default
<code>database/MySQLClient.ts</code>	mysql2 — not installed by default
<code>database/MsSqlClient.ts</code>	mssql — not installed by default
<code>database/OracleClient.ts</code>	oracledb — not installed by default

runtime

`src/core/runtime/`

The wiring that makes dependency injection work. `fixtures.ts` is the single file every test imports from; `hooks/` run setup/teardown and failure capture; `listeners/` attach console/network logging and the custom pass/fail summary reporter.

<code>fixtures/fixtures.ts</code>	every injectable: pages, services, config, logger...
<code>hooks/GlobalSetup.ts</code>	creates report dirs, writes Allure environment.properties
<code>hooks/GlobalTeardown.ts</code>	
<code>hooks/TestLifecycleHooks.ts</code>	captureOnFailure — screenshot + last 50 log lines
<code>listeners/PageEventListener.ts</code>	console/network logging, attached per test
<code>listeners/CustomReporter.ts</code>	structured pass/fail/skip/duration summary via Winston

data

[src/core/data/](#)

Test data and the shapes it takes. `builders/` are Fluent builders
(`.withEmail().withPassword().build()`); `testdata/` holds static JSON/CSV
plus typed endpoint/payload catalogs; `models/` and `enums/` are the shared
domain vocabulary.

```
builders/UserBuilder.ts
builders/JobBuilder.ts
testdata/urls.testdata.ts
testdata/apiEndpoints.testdata.ts
testdata/payloads.testdata.ts
testdata/users.testdata.ts
testdata/sample-users.json / .csv
models/User.model.ts / Job.model.ts / ApiResponse.model.ts
enums/BrowserType.ts, Environment.ts, DatabaseType.ts, UserRole.ts,
    Status.ts, Country.ts, Language.ts, ApplicationModule.ts
```

config

[src/core/config/](#)

`Config` is a Singleton — typed getters over whichever `.env.<tier>` loaded,
with sane fallback defaults so the demo app works with zero setup. Nothing in
the framework reads `process.env` directly outside this file.

<code>Config.ts</code>	<code>environment, baseUrl, apiBaseUrl, adminUsername/Password, headless, retries, browser, dbType...</code>
<code>constants/EnvKeys.ts</code>	the <code>.env</code> variable names, as constants
<code>constants/Routes.ts</code>	<code>/login, /dashboard, /users, /products, /reports</code>
<code>constants/Timeouts.ts</code>	<code>DEFAULT, SHORT, MEDIUM, LONG, PAGE_LOAD, API_REQUEST...</code>
<code>constants/Messages.ts, Roles.ts, ReportNames.ts, FilePaths.ts</code>	
<code>logger/Logger.ts</code>	Winston — daily file + error file + console (dev only)

shared

[src/core/shared/](#)

Cross-cutting support with no opinion about pages, API, or data specifically. This
is also where every `*Utils` static helper lives — 20 of them, including four
purpose-built ones added alongside this guide: `AllureUtils`,
`AccessibilityUtils`, `PerformanceUtils`, and `VisualRegressionUtils`.

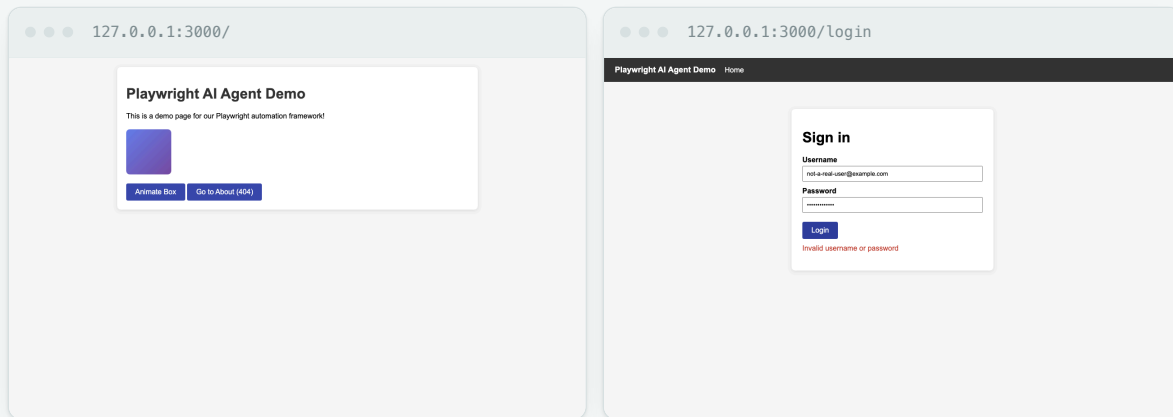
<code>validations/ValidationHelpers.ts</code>	<code>verifyText, verifyUrl, verifyElementVisible...</code>
<code>exceptions/</code>	<code>APIError, ConfigurationError, DatabaseError, ElementError, ValidationError, FrameworkError</code>
<code>helpers/TestDataProvider.ts</code>	<code>readJson/readCsv/readExcel</code> from <code>data/testdata</code>
<code>helpers/RetryHelper.ts, ReportAttachmentHelper.ts</code>	
<code>interfaces/IConfig.ts</code>	
<code>utils/AllureUtils.ts</code>	<code>step/steps/attachScreenshot/attachVideo/setCategory</code>
<code>utils/AccessibilityUtils.ts</code>	<code>runAxeScan(page)</code> — shared axe-core defaults
<code>utils/PerformanceUtils.ts</code>	Lighthouse audit (lazy-loaded ESM import) + page-load metrics
<code>utils/VisualRegressionUtils.ts</code>	per-project-namespaced screenshot capture/compare fixture
<code>utils/BrowserUtils.ts</code>	builds the 5 Playwright projects
<code>utils/ScreenshotUtils.ts, DateUtils, JsonUtils, CSVUtils, ExcelUtils, FileUtils, NetworkUtils, RandomDataUtils, StorageUtils, DownloadUtils, UploadUtils, EncryptionUtils, ZipUtils, APIUtils, WaitUtils</code>	
<code>tools/dev-server.js, compare.js, save-chatmode-response.js</code>	demo server, visual-diff CLI, chatmode response archiver (see be

Every one of these is a plain static-method class named `XxxUtils` — that's the convention. Adding a new cross-cutting helper means adding one file here, not a new pattern.

The bundled demo app

A real target to point every example in this guide at.

Served from `demo/` by a tiny static server (`src/core/tools/dev-server.js`) on **`http://127.0.0.1:3000`**. Playwright starts and stops it automatically around every test run — you never launch it by hand unless you're exploring manually with `npm run dev:server` .



Home — the target for `DemoPage` and the smoke suite.

Login, mid-validation — the target for `LoginPage` .

The four page objects

All of `src/pages/` — locators, methods, and what each one is standing in for.

LoginPage.ts

`usernameInput · passwordInput · loginButton · errorMessage · header`

Composes one component (`HeaderComponent`). Methods:

`navigate()`, `enterUsername()`, `enterPassword()`, `clickLogin()`, `login()` (does all three), `getErrorMessage()`, `getErrorMessageLocator()`.

DemoPage.ts

`animatedBox · animateButton · navButton`

The simplest page in the repo — no composed components.

Methods: `goToDemo()`, `clickAnimateButton()`, `clickNavButton()`, `sampleAnimationFrames(durationMs)`.

WeSendCVPPage.ts

`emailInput · passwordInput · submitButton · navigationMenu`

The one page object that targets a real external site (`wesendcv.com`) rather than the bundled demo. Methods:

`goToWeSendCV(url)`, `login()`, `getNavigationMenu()`.

DashboardPage.ts

6 composed components + `saveButton`

The reference example for the Component Object Model — see below.

Composition in practice

`DashboardPage` doesn't declare its own table/pagination/toast locators — it composes six independent components, each already tested and reused elsewhere:

```
export class DashboardPage extends BasePage {
  public readonly header: HeaderComponent;
  public readonly navigationMenu: NavigationMenu;
  public readonly jobsTable: TableComponent;
  public readonly pagination: Pagination;
  public readonly toast: ToastMessage;
  public readonly logoutDialog: CommonDialog;

  constructor(page: Page) {
    super(page);
    this.header = new HeaderComponent(page);
    this.jobsTable = new TableComponent(page, page.getByRole('table', { name: 'Jobs' }));
    this.pagination = new Pagination(page, page.getByRole('navigation', { name: 'Pagination' }));
    this.toast = new ToastMessage(page);
    this.logoutDialog = new CommonDialog(page);
  }

  async saveChanges(): Promise<void> {
    await this.click(this.saveButton);
  }
}
```

Dashboard

Title	Location	Salary
QAAutomation Engineer	Remote	\$95,000
Senior SDET	Austin, TX	\$130,000
Backend Engineer	New York, NY	\$140,000

[Previous](#) Page 1 of 2 [Next](#)

Save Changes

Saved successfully

Every visible piece here is a separate, reusable component: `header` (top bar), `jobsTable` (the grid), `pagination` (Previous / Page 1 of 2 / Next), and `toast` — captured mid-confirmation after calling `saveChanges()`.

A component never knows which page it's on. `TableComponent` is generic over any `role="table"` — it's used by `DashboardPage` here, but nothing about it is dashboard-specific. That's what makes it reusable instead of copy-pasted.

Running tests

Every command you'll actually type.

By category

COMMAND	RUNS
<code>npm test</code>	Everything, across all five browser projects
<code>npm run test:smoke</code>	Critical-path smoke tests
<code>npm run test:sanity</code>	Quick PR sanity checks
<code>npm run test:regression</code>	Full regression suite
<code>npm run test:e2e</code>	End-to-end user flows
<code>npm run test:api</code>	API-only tests

Browser & environment

COMMAND	EFFECT
<code>npx playwright test --project=chromium</code>	One browser only
<code>BROWSER=firefox npm test</code>	Pin the whole run to one engine
<code>ENVIRONMENT=stage npm test</code>	Load <code>.env.stage</code> instead of <code>.env.qa</code>

Debugging

MODE	COMMAND
Watch it run headed	<code>npm run test:headed</code>
Step through with the Inspector	<code>npm run test:debug</code>
Interactive UI mode	<code>npm run test:ui</code>

Five browser projects run by default: **chromium**, **firefox**, **webkit**, plus **Mobile Chrome** and **Mobile Safari** emulation — configured once in `BrowserUtils.buildProjects()`, not repeated per test.

Tutorial: write a script the right way

Best practices first, then one real script built and run from scratch.

Best practices, as code

The Coding Standards below aren't a style preference — each one exists because the alternative broke something. Here's what following (and breaking) each one actually looks like:

DO

- Extend `BasePage` for every action
- Import `{ test, expect }` from `fixtures.ts`
- Destructure fixtures: `async ({ dashboardPage })`
- Assert via `ValidationHelpers` or `expect()` in the test
- Pull values from `Config` / `constants`
- `getByRole` / `getByLabel` / `getByTestId`

DON'T

- Call `page.click(...)` directly in a page object
- Import from `@playwright/test` in a spec
- `new DashboardPage(page)` inside a test
- Assert inside a page object method
- Hardcode a URL, password, or timeout number
- CSS selectors, and **never** XPath

One rule earns its own line: **a page object that asserts is the single most common review comment in this codebase**. If a method name starts to want a `toBe` / `toHave` inside it, that logic belongs in the test or in `ValidationHelpers` instead.

Worked example: dashboard save confirmation

Goal: verify that clicking **Save Changes** shows the "Saved successfully" toast — using only what already exists (`dashboardPage` is already a registered fixture, `saveChanges()` and `toast` already exist on it), so this is purely a new spec file.

1 Check what's already available

Before writing anything, look at the page object. `DashboardPage` already exposes `saveChanges()` and a composed `toast` — nothing new to build.

2 Write the spec

Categorize it with `AllureUtils.setCategory`, name each phase with `AllureUtils.step`, and attach proof with `AllureUtils.attachScreenshot` — on pass, not just on failure.

```
import { test, expect } from '../../../src/core/runtime/fixtures/fixtures';
import { AllureUtils } from '../../../src/core/shared/utils/AllureUtils';

test.describe('Dashboard Save Confirmation', () => {
  test.beforeEach(async () => {
    await AllureUtils.setCategory('Dashboard', 'Save Changes', AllureUtils);
  });

  test('shows a success toast after saving changes', async ({ dashboardPage }) => {
    await AllureUtils.step('Navigate to the dashboard', async () => {
      await dashboardPage.navigate();
    });

    await AllureUtils.step('Click Save Changes', async () => {
      await dashboardPage.saveChanges();
    });

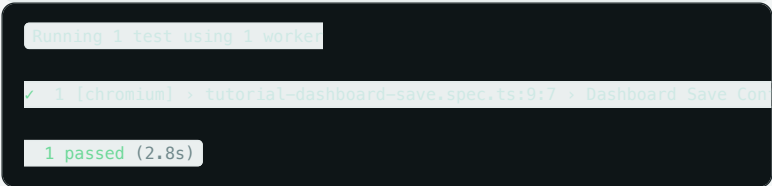
    await AllureUtils.step('Verify the success toast appears', async () => {
      await dashboardPage.toast.waitForVisible();
      expect(await dashboardPage.toast.getMessage()).toContain('Saved successfully');
    });

    await AllureUtils.attachScreenshot(page, 'Toast confirmation');
  });
});
```

3

Run it

```
npx playwright test tests/smoke/tutorial-dashboard-save.spec.ts --project=chromium
```



```
Running 1 test using 1 worker
✓ 1 [chromium] › tutorial-dashboard-save.spec.ts:9:7 › Dashboard Save Confirmation
1 passed (2.8s)
```

That's the actual, unedited output from running this exact file against this exact repo.

4

Check the evidence

Regenerate the Allure report (`npm run allurereport`) — the run now has its own named steps and a screenshot attached, in the same style as every example in the [AllureUtils](#) section above.

Nothing here required touching a page object. That's the point of the pattern: most new tests are new specs against existing fixtures. You only add to `src/pages/` or `src/core/` when a genuinely new interaction doesn't exist yet — see the next section for that case.

When you *do* need a new page object

1

Extend BasePage

Locators + actions + navigation only — no assertions, ever.

```
export class DemoPage extends BasePage {
  private readonly animateButton: Locator;

  constructor(page: Page) {
    super(page);
    this.animateButton = this.page.getByRole('button', { name: /animate
  }

  async clickAnimateButton(): Promise<void> {
    await this.click(this.animateButton);
  }
}
```

2 Register it as a fixture

In `src/core/runtime/fixtures/fixtures.ts` — this is what makes it injectable instead of hand-instantiated.

```
demoPage: async ({ page }, use) => {
  await use(new DemoPage(page));
},
```

3 Write the spec, run it, done

Same shape as the worked example above — fixtures in, `AllureUtils` around every phase, no `new`, no assertions in the page object.

09

8 sample scripts, real reporting patterns

`tests/zsampleScript/` — a working, passing reference for the framework's core test disciplines.

Each file is a real spec, not pseudo-code — it runs against the demo app as part of a plain `npm test`, and demonstrates the pattern in miniature.

```
npx playwright test tests/zsampleScript # the whole folder
npx playwright test tests/zsampleScript/security-tests.sample.spec.ts # one category
```

Integration

01

Multi-step workflows across components.

[integration-tests.sample.spec.ts](#)

Performance

02

Load time, First Contentful Paint, resource count.

[performance-tests.sample.spec.ts](#)

Security

03

Blocked auth, XSS injection, header validation.

[security-tests.sample.spec.ts](#)

Validation

04

Format checks, length limits, malicious patterns.

[validation-tests.sample.spec.ts](#)

Mock

05

Stubbed failures, slow responses, dead services.

[mock-tests.sample.spec.ts](#)

Accessibility

06

Axe scan, keyboard-only nav, focus order.

[accessibility-tests.sample.spec.ts](#)

E2E

07

Full critical-path journey, entirely via POM.

[e2e-tests.sample.spec.ts](#)

Visual regression

08

Per-project-baseline pixel diffing plus page-load metrics.

[visual-regression.sample.spec.ts](#)

Reporting

Two independent reports come out of every run — a fast native one, and a detailed one.

Playwright's own HTML report

Zero setup — every run writes it. Best for a quick pass/fail scan right after a local run.

The screenshot shows a web browser window titled "reports/html - npx playwright show-report". The interface includes a search bar, a summary bar with statistics (All 39, Passed 39, Failed 0, Flaky 0, Skipped 0), and a table of test results. The tests are grouped by spec file.

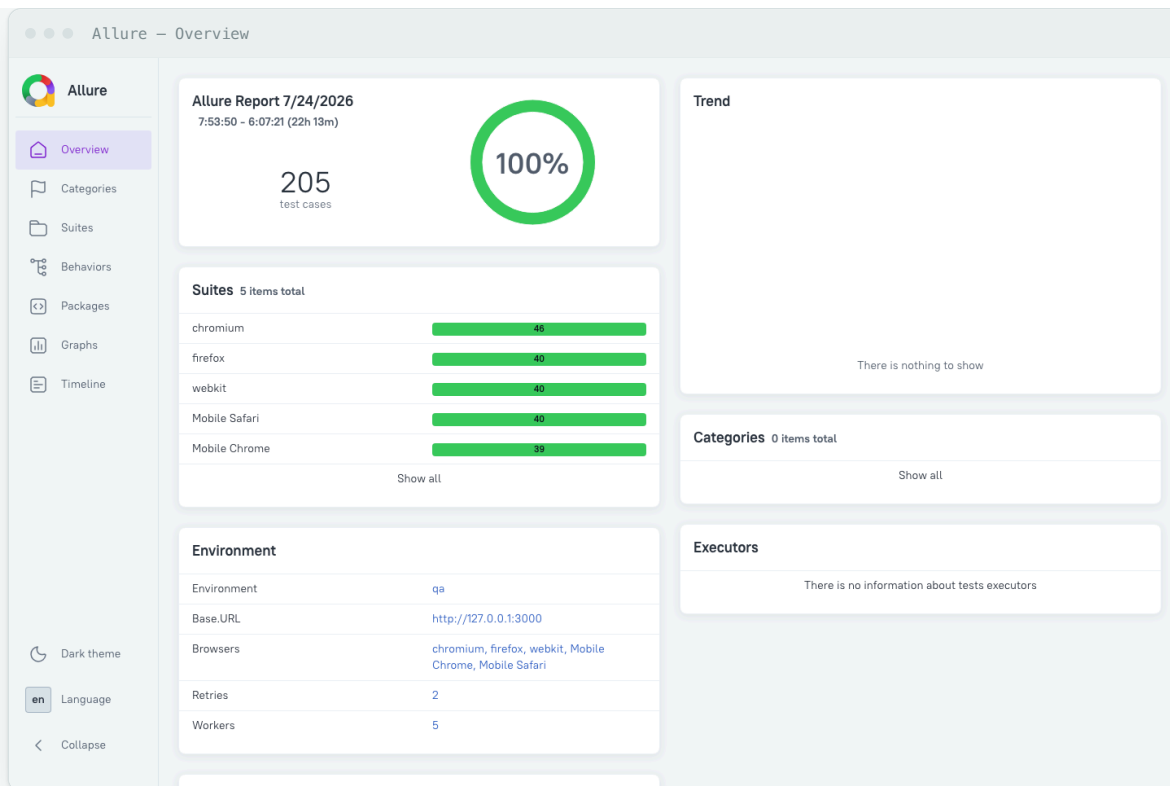
Spec File	Test Name	Duration	Status
smoke/demo-smoke.spec.ts	Demo Page - Smoke Tests › should load demo page successfully	295ms	Passed
	Demo Page - Smoke Tests › should click animate button	356ms	Passed
zsampleScript/accessibility-tests.sample.spec.ts	Sample - Accessibility Tests › page has no automatically detectable a11y violations (Axe accessibility checks)	349ms	Passed
	Sample - Accessibility Tests › interactive elements are reachable via keyboard alone (keyboard-only navigation)	306ms	Passed
	Sample - Accessibility Tests › focus order is sequential and does not get trapped (focus order)	553ms	Passed
zsampleScript/chaos-tests.sample.spec.ts	Sample - Chaos Tests › multiple concurrent users can load the app at the same time (multiple users)	196ms	Passed
	Sample - Chaos Tests › one user surviving a backend outage while another succeeds (DB/backend failures)	171ms	Passed
	Sample - Chaos Tests › requests completing in a random order do not corrupt page state (random delays)	94ms	Passed
zsampleScript/e2e-tests.sample.spec.ts	Sample - E2E Tests › critical path: a user attempts to sign in end-to-end via the POM	351ms	Passed

Grouped by spec file, with duration and browser project per test.

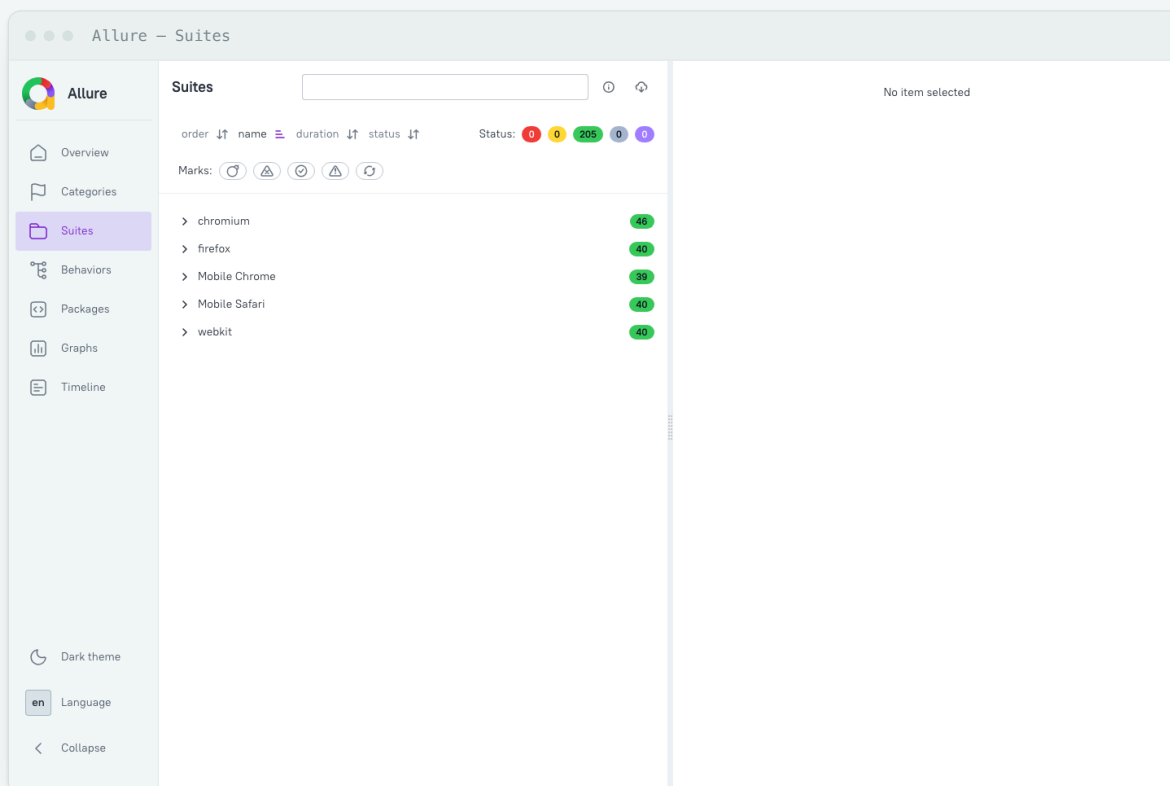
Allure — the detailed report

Raw results are written automatically on every run (the `allure-playwright` reporter is already registered). Generating and opening the static site is a separate step:

```
npm run allurereport # generate + open in one command
# - or, separately -
npm run allure:generate
npm run allure:open
```



The Environment panel is populated automatically by `GlobalSetup.ts` — every report says exactly which `.env` tier, browsers, and retry count produced it.



Every browser project gets its own suite tree — nothing to configure.

Named steps and evidence, on every run

Not just on failure — `AllureUtils` attaches proof for every test, pass or fail.

Playwright's own screenshot/video capture is **failure-only** by config. `AllureUtils` (`src/core/shared/utils/AllureUtils.ts`) is the fix: a small façade that wraps `allure-js-commons` so a spec writes one line instead of five, and gets a named, collapsible step with a real screenshot inside it regardless of outcome.

The screenshot displays the Allure Test body interface. On the left is a sidebar with navigation links: Overview, Categories, Suites (selected), Behaviors, Packages, Graphs, and Timeline. Below these are settings for Dark theme and Language (en). The main area is titled 'Suites' and shows a search bar with 'invalid credentials never reach'. Below the search bar are filters for order, name, duration, and status. The status bar shows 0 red, 0 yellow, 205 green, 0 blue, and 0 purple. The test suite is expanded, showing a tree structure of tests. The selected test is '#3 invalid credentials never reach the authenticated area (authentication)' with a duration of 351ms. The right panel shows the test details, including a 'Passed' status, severity of 'blocker', duration of 351ms, and a list of steps: Before Hooks (8 sub-steps, 62ms), Navigate to login and attempt sign-in (7 sub-steps, 103ms), Screenshot (20ms), Verify access was denied (3 sub-steps, 24ms), Login attempt result (21.8 KiB), and After Hooks (6 sub-steps, 149ms).

Severity, duration, and a full step timeline — generated by three `AllureUtils` calls in the spec.

The screenshot shows the Allure report interface. On the left is a sidebar with navigation links: Overview, Categories, Suites (selected), Behaviors, Packages, Graphs, and Timeline. The main area is divided into two panels. The left panel shows a tree view of test suites and steps. The right panel shows a detailed view of a specific step, including its severity, duration, parameters, and a screenshot of the test failure.

Suites

invalid credentials never reach

order ↑ name ↓ duration ↑ status ↓ Status: 0 0 205 0 0

Marks: [Icons]

- chromium
 - zsampleScript/security-tests.sample.spec.ts
 - Sample - Security Tests
 - #3 invalid credentials never reach the authenticated area (authentication) 351ms
- firefox
 - zsampleScript/security-tests.sample.spec.ts
 - Sample - Security Tests
 - #1 invalid credentials never reach the authenticated area (authentication) 740ms
 - Mobile Chrome
 - zsampleScript/security-tests.sample.spec.ts
 - Sample - Security Tests
 - #2 invalid credentials never reach the authenticated area (authentication) 376ms
 - Mobile Safari
 - zsampleScript/security-tests.sample.spec.ts
 - Sample - Security Tests
 - #1 invalid credentials never reach the authenticated area (authentication) 633ms

Test body

 - Before Hooks 8 sub-steps 62ms
 - Navigate to login and attempt sign-in 7 sub-steps 103ms
 - Screenshot 20ms
 - Verify access was denied 3 sub-steps 24ms
 - Login attempt result 21.8 KiB
 - After Hooks 6 sub-steps 149ms

Sign in

Username: test@mail.com@example.com

Password: [Redacted]

Sign in

Invalid username or password

The screenshot is inline, at the exact step it was taken — not bundled separately at the end.

The API

`AllureUtils.setCategory(epic, feature, severity?)`

Sets the labels a test is grouped and prioritized by in one call, instead of three separate `allure.epic/feature/severity` calls.

`AllureUtils.step(name, fn)`

Wraps a block of actions as one named, collapsible step — the building block for everything else here.

`AllureUtils.steps(name, ...actions)`

Same idea, for a flat list of actions run in sequence — skips writing `async () => { await a(); await b(); } at the call site.`

`AllureUtils.attachScreenshot(page, name?)`

Captures and attaches a PNG, regardless of pass or fail — this is what produced both screenshots above.

`AllureUtils.attachJson(name, data)`

Attaches any object as pretty-printed JSON — request payloads, computed values, anything worth inspecting later.

`AllureUtils.attachVideo(page, context, name?)`

Attaches the full recording for a test. Requires a context created with `recordVideo` set — see `e2e-tests.sample.spec.ts` for the complete pattern.

`AllureUtils.parameter(name, value)`

Shows a labelled value on the report page — durations, computed flags, anything worth surfacing without a full attachment.

Regenerate after every run. Raw results write automatically, but the static site does not rebuild itself — run `npm run allurereport` (or `allure:generate`) after the tests finish, every time, or you'll be looking at a stale report.

Visual regression

Pixel-level diffing, namespaced per browser project.

`src/core/tools/compare.js` diffs two PNGs with `pixelmatch` and fails past a configurable threshold. If no baseline exists yet, it creates one from the current screenshot and exits clean — the first run is never a false failure.

```
npm run compare -- baseline.png current.png diff.png --threshold=0.03
```

The same technique is also available as a typed, importable method —

`ScreenshotUtils.compare()` in `src/core/shared/utils/` — for specs that want the check without shelling out to a script.

For a fixture-driven version wired straight into a spec, use the `visualRegressionUtils` fixture (`VisualRegressionUtils` , `src/core/shared/utils/VisualRegressionUtils.ts`):

```
test('Demo Page Visual Test', async ({ demoPage, visualRegressionUtils }) => {
  await demoPage.goToDemo();
  const matches = await visualRegressionUtils.checkVisualRegression('demo-page', {
    expect(matches).toBeTruthy();
  });
});
```

Baselines are namespaced per browser project —

`reports/visual/{baseline,actual,diff}/<project>/` , keyed off `testInfo.project.name` . Without this, chromium/firefox/webkit/Mobile Chrome/Mobile Safari would all race to write and compare against one shared baseline file, and every engine except whichever ran first would fail permanently on real (and expected) cross-engine rendering differences.

Chatmodes & skills

Five personas you switch into, two behaviors Copilot applies on its own.

NAME	KIND	BEST FOR
 Healer	chatmode	Diagnose and fix a failing test
 Planner	chatmode	Draft a test plan before writing code
 Generator	chatmode	Write spec files and page objects
 API Testing	chatmode	Scaffold API and Pact contract tests
 Manual Testing	chatmode	Generate a manual QA checklist
 Code Review	skill · auto	Audit POM compliance, security, anti-patterns
 Test Debugging	skill · auto	Systematic failure triage

Activating a chatmode in VS Code

- 1 Open Copilot Chat**
`Cmd/Ctrl+Alt+I`
- 2 Switch the mode dropdown to Agent**
 Chatmodes only get file-write/command-execution tools in Agent mode — from Ask mode a chatmode can talk but can't create files or run tests, regardless of what its instructions say.
- 3 Pick from the dropdown**
 Or skip it and prefix your message directly: `@healer Fix the failing test in tests/wesendcv/wesendcv.spec.ts`
- 4 Skills need nothing**
 Just ask a normal question — "review this spec for POM compliance" loads the Code Review skill automatically.

Every response is archived. Each chatmode runs `src/core/tools/save-chatmode-response.js` (via the terminal tool) after every reply, saving a timestamped copy to `docs/chatmode-responses/<chatmode>--<topic>--<YYYYMMDDTHHMMSSZ>.md` — a plan or diagnosis survives after the chat session ends, and a revision gets its own file instead of overwriting the last one.

MCP server

Let an AI client run your tests as tool calls instead of shell commands.

`mcp/mcp-server.ts` exposes this framework over the Model Context Protocol (stdio transport). Any MCP-compatible client — Claude Desktop, Claude Code, VS Code — can launch it as a subprocess.

```
npm run mcp:server
```

TOOL	ARGUMENTS	DOES
<code>run_playwright_test</code>	testFile, project?, headed?	Runs one spec file
<code>run_all_tests</code>	—	Runs the full suite
<code>get_test_report</code>	—	Lists <code>reports/html/</code> contents

CLIENT CONFIG

```
{
  "mcpServers": {
    "playwright-ai-agent": {
      "command": "npx",
      "args": ["tsx", "mcp/mcp-server.ts"],
      "cwd": "/absolute/path/to/repo"
    }
  }
}
```

Running in Docker

Same suite, containerized — for parity with CI.

```
docker compose up --build
```

Reports and results are bind-mounted back to the host, so `reports/` is populated exactly as a local run would leave it. Override the default command to run a specific suite:

```
docker compose run --rm playwright-tests npm run test:smoke
```

CI/CD

What happens on every push and PR.

`.github/workflows/ci.yml` runs on every push and pull request against `main` :

```
checkout → setup-node(20) → npm ci → lint → typecheck
→ install:browsers → npm test → allure:generate → upload artifact
```

A manual `workflow_dispatch` trigger lets you pick the environment and browser for an on-demand run from the Actions tab.

Troubleshooting

The fast paths, in order of how often you'll need them.

A test failed

Check `reports/html` first — screenshot, video, and trace are already attached, plus the last 50 log lines.

Need to step through it

`npm run test:debug` for the Inspector, or `npm run test:ui` for the interactive runner.

Allure report looks stale

Regenerate — it never rebuilds itself. `npm run allurereport`.

Browser install issues

`npm run install:browsers` — safe to re-run any time.

Generated from the live repository — every command, file path, and screenshot on this page was run and captured directly against this codebase.